

ETI – Aufgabenblatt 5

Hausaufgaben

Aufgabe 5.6

Aufgabenstellung.

Bringen Sie die Grammatik

$$S \rightarrow SaSb \mid A,$$

$$A \rightarrow \varepsilon \mid B \mid a,$$

$$B \rightarrow bB \mid A \mid C,$$

$$C \rightarrow c$$

in Chomsky-Normalform. Das Startsymbol ist S .

Methodik / Definition.

Eine CFG ist in Chomsky-Normalform, wenn jede Regel eine der Formen

$$A \rightarrow BC, \quad A \rightarrow a, \quad S_0 \rightarrow \varepsilon$$

hat. Die letzte Regel ist nur fuer die Startvariable erlaubt, wenn ε in der Sprache liegt. Damit die Startvariable nicht rechts vorkommt, fuehrt man zuerst eine neue Startvariable ein. Danach entfernt man ε -Regeln, Kettenregeln und ersetzt Terminale in laengeren rechten Seiten durch eigene Variablen.

In den Vergleichstabellen markiert **blau**, was neu hinzukommt oder ersetzt wird. **Rot** markiert Regeln, die in diesem Schritt verschwinden.

Schritt 1: Neue Startvariable.

Da S rechts in $S \rightarrow SaSb$ vorkommt und ε erzeugt werden kann, fuehren wir eine neue Startvariable S_0 ein:

$$S_0 \rightarrow S.$$

Schritt 2: ε -Regeln entfernen.

Nullable sind A , B und S , denn

$$A \Rightarrow \varepsilon, \quad B \Rightarrow A \Rightarrow \varepsilon, \quad S \Rightarrow A \Rightarrow \varepsilon.$$

Aus $S \rightarrow SaSb$ entstehen daher alle Varianten, in denen die beiden Vorkommen von S optional gestrichen werden. Nur fuer die neue Startvariable behalten wir ε :

Vorher	Nachher ohne ε -Regeln
$S_0 \rightarrow S$	$S_0 \rightarrow S \mid \varepsilon$
$S \rightarrow SaSb \mid A$	$S \rightarrow SaSb \mid aSb \mid Sab \mid ab \mid A$
$A \rightarrow \varepsilon \mid B \mid a$	$A \rightarrow B \mid a$
$B \rightarrow bB \mid A \mid C$	$B \rightarrow bB \mid b \mid A \mid C$
$C \rightarrow c$	$C \rightarrow c$

Schritt 3: Kettenregeln entfernen.

Die Kettenregeln sind $S_0 \rightarrow S$, $S \rightarrow A$, $A \rightarrow B$, $B \rightarrow A$ und $B \rightarrow C$. Wir ersetzen sie, indem wir die nichttrivialen Regeln der erreichbaren Variablen direkt uebernehmen:

Vorher	Nachher ohne Kettenregeln
$S_0 \rightarrow \varepsilon \mid S$	$S_0 \rightarrow \varepsilon \mid SaSb \mid aSb \mid Sab \mid ab \mid a \mid bB \mid b \mid c$
$S \rightarrow SaSb \mid aSb \mid Sab \mid ab \mid A$	$S \rightarrow SaSb \mid aSb \mid Sab \mid ab \mid a \mid bB \mid b \mid c$
$A \rightarrow B \mid a$	$A \rightarrow a \mid bB \mid b \mid c$
$B \rightarrow bB \mid b \mid A \mid C$	$B \rightarrow bB \mid a \mid b \mid c$
$C \rightarrow c$	$C \rightarrow c$

Die Variablen A und C sind nun vom Startsymbol aus nicht mehr erreichbar und koennen entfernt werden.

Schritt 4: Terminale in langen Regeln ersetzen und binaer machen.

Wir fuehren

$$T_a \rightarrow a, \quad T_b \rightarrow b$$

ein. Fuer die langen rechten Seiten nutzen wir zusaetzlich

$$E \rightarrow T_a T_b, \quad G \rightarrow S T_b, \quad F \rightarrow T_a G.$$

Damit erhalten wir die CNF-Grammatik

$$G_{\text{CNF}} = (V', \{a, b, c\}, R', S_0)$$

mit

$$V' = \{S_0, S, B, T_a, T_b, E, F, G\}$$

und Regeln

Vorher	Nachher in CNF
$S_0 \rightarrow \varepsilon \mid SaSb \mid aSb \mid Sab \mid ab \mid a \mid bB \mid b \mid c$	$S_0 \rightarrow \varepsilon \mid SF \mid T_a G \mid SE \mid T_a T_b \mid a \mid T_b B \mid b \mid c$
$S \rightarrow SaSb \mid aSb \mid Sab \mid ab \mid a \mid bB \mid b \mid c$	$S \rightarrow SF \mid T_a G \mid SE \mid T_a T_b \mid a \mid T_b B \mid b \mid c$
$B \rightarrow bB \mid a \mid b \mid c$	$B \rightarrow T_b B \mid a \mid b \mid c$
	$E \rightarrow T_a T_b$
	$F \rightarrow T_a G$
	$G \rightarrow S T_b$
	$T_a \rightarrow a$
	$T_b \rightarrow b$

Alle rechten Seiten haben nun die Form XY , ein einzelnes Terminal oder, nur bei S_0 , das Wort ε . Ausserdem kommt S_0 auf keiner rechten Seite vor.

Aufgabe 5.7

Aufgabenstellung.

Zeigen Sie, dass die kontextfreien Sprachen unter Vereinigung, Konkatenation und Kleene-Stern abgeschlossen sind. Geben Sie jeweils eine Konstruktionsvorschrift fuer die entsprechende Grammatik an.

Methodik / Definition.

Eine Sprache ist kontextfrei, wenn es eine CFG gibt, die sie erzeugt. Fuer Abschlussbeweise reicht es daher, aus gegebenen CFGs neue CFGs zu konstruieren, die genau die gewuenschte zusammengesetzte Sprache erzeugen.

Seien L_1, L_2 kontextfrei. Dann gibt es CFGs

$$G_1 = (V_1, \Sigma, R_1, S_1), \quad G_2 = (V_2, \Sigma, R_2, S_2)$$

mit $L(G_1) = L_1$ und $L(G_2) = L_2$. Wir nehmen ohne Einschraenkung an, dass $V_1 \cap V_2 = \emptyset$ ist, denn Variablen kann man umbenennen.

Vereinigung. Definiere

$$G_{\cup} = (V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \rightarrow S_1 \mid S_2\}, S).$$

Dann waehlt die erste Regel genau aus, ob ein Wort mit G_1 oder mit G_2 erzeugt wird. Also gilt

$$L(G_{\cup}) = L_1 \cup L_2.$$

Konkatenation. Definiere

$$G_{\circ} = (V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}, S).$$

Jede Ableitung erzeugt zuerst ein Wort aus L_1 und danach ein Wort aus L_2 . Umgekehrt kann jedes uv mit $u \in L_1$ und $v \in L_2$ so abgeleitet werden. Also gilt

$$L(G_{\circ}) = L_1 L_2.$$

Kleene-Stern. Fuer L_1^* definieren wir

$$G_* = (V_1 \cup \{S\}, \Sigma, R_1 \cup \{S \rightarrow S_1 S \mid \varepsilon\}, S).$$

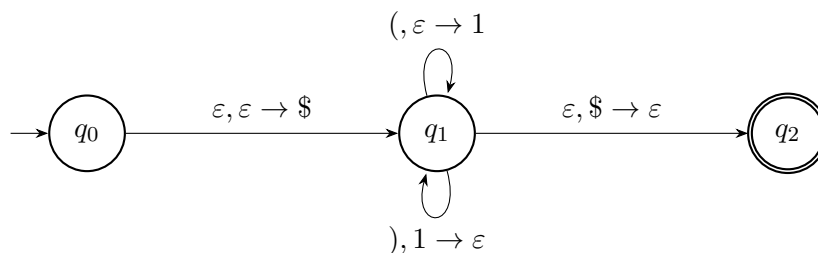
Die Regel $S \rightarrow \varepsilon$ erzeugt null Faktoren. Jede Anwendung von $S \rightarrow S_1 S$ fuegt vorne ein weiteres Wort aus L_1 hinzu. Daher erzeugt die Grammatik genau endliche Konkatenationen von Woertern aus L_1 , also

$$L(G_*) = L_1^*.$$

Aufgabe 5.8

Aufgabenstellung.

Bestimmen Sie die Sprache, die von dem gegebenen PDA akzeptiert wird, und geben Sie eine moeglichst intuitive Beschreibung an.



Methodik / Definition.

Bei einem PDA bedeutet die Kantenbeschriftung $a, b \rightarrow c$: Lies a , entferne b vom Stack und lege c auf den Stack. Falls ein Eintrag ε ist, wird an dieser Stelle nichts gelesen, entfernt oder hinzugefuegt. Ein nichtdeterministischer PDA akzeptiert, wenn mindestens ein Berechnungspfad in einem akzeptierenden Zustand endet, nachdem die Eingabe gelesen wurde.

Der PDA legt zuerst einen Bodenmarker $\$$ auf den Stack. Danach gilt im Zustand q_1 :

$(, , \varepsilon \rightarrow 1$ und $), 1 \rightarrow \varepsilon$.

Also wird fuer jede oeffnende Klammer ein 1 auf den Stack gelegt, und fuer jede schliessende Klammer wird ein 1 entfernt. Akzeptiert wird genau dann, wenn am Ende wieder nur der Bodenmarker uebrig ist.

Formal zaehlen wir fuer ein Wort u die Anzahl der oeffnenden und schliessenden Klammern mit $|u|_(<$ bzw. $|u|_>$. Schreibe $u \preceq w$, wenn u ein Praefix von w ist. Dann ist die akzeptierte Sprache

$$L(P) = D$$

mit

$$D = \{w \in \{(<, >\}^* \mid \forall u \preceq w : |u|_(< \geq |u|_> \text{ und } |w|_(< = |w|_>\}.$$

Begrueundung: Nach dem Lesen eines Praefixes u liegt ueber dem Bodenmarker genau $|u|_(< - |u|_>$ -mal das Symbol 1 auf dem Stack, solange dieser Wert nicht negativ wird. Wird er negativ, muesste der Automat bei

einer schliessenden Klammer ein 1 entfernen, das nicht vorhanden ist; dieser Berechnungspfad bricht also ab. Der Uebergang nach q_2 ist genau dann moeglich, wenn keine 1 mehr ueber dem Bodenmarker liegen, also wenn $|w|_< = |w|_>$ gilt.

Intuitiv ist das die Sprache der korrekt geklammerten Woerter mit genau einer Klammerart.

Beispiele.

Akzeptiert werden zum Beispiel

$\varepsilon, \quad (), \quad (()), \quad ()()$.

Nicht akzeptiert werden zum Beispiel

$)(), \quad ((, \quad ()).$

Aufgabe 5.9

Aufgabenstellung.

Entwerfen Sie einen PDA fuer

$$L = \{a^i b^j c^k \mid 2j = 3k \vee i + k = j\}$$

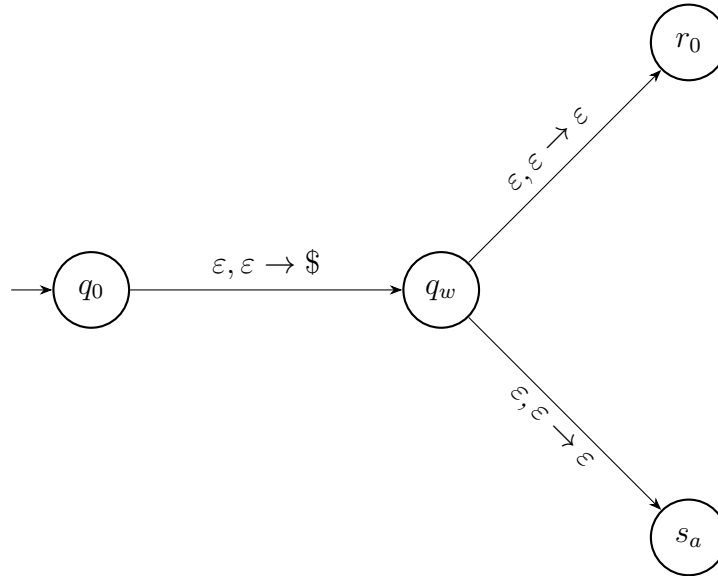
und begruenden Sie kurz, weshalb der PDA die Sprache erkennt.

Methodik / Definition.

Da PDAs nichtdeterministisch sind, kann man fuer eine Vereinigung zweier Bedingungen zuerst nichtdeterministisch einen Zweig waehlen. Der eine Zweig prueft $2j = 3k$, der andere Zweig prueft $i + k = j$.

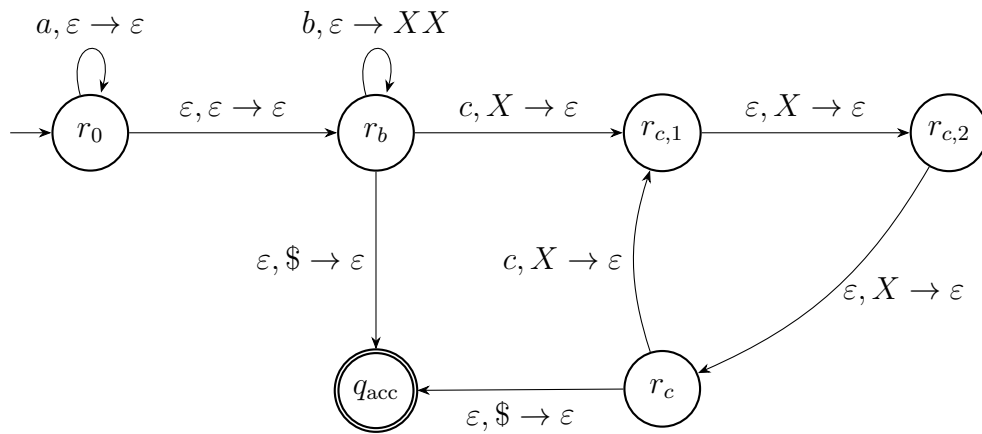
Wir beschreiben einen PDA P ueber dem Eingabealphabet $\{a, b, c\}$. Zuerst legt P den Bodenmarker $\$$ auf den Stack und waehlt dann nichtdeterministisch einen der beiden folgenden Zweige.

Gemeinsamer Start und nichtdeterministische Wahl.



Zweig 1: Pruefe $2j = 3k$

Die a 's werden ignoriert. Fuer jedes gelesene b werden zwei Symbole X auf den Stack gelegt. Fuer jedes gelesene c werden drei Symbole X entfernt. Am Ende wird akzeptiert, wenn genau der Bodenmarker erreicht ist.

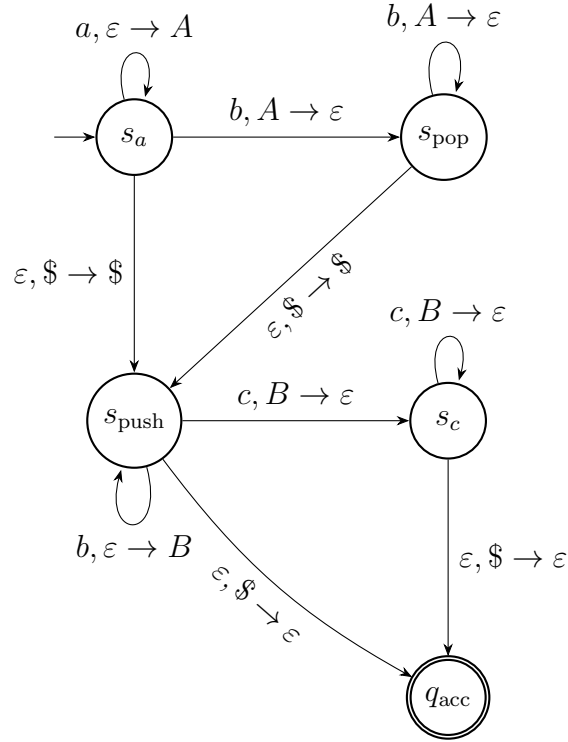


Dieser Zweig akzeptiert genau dann, wenn die Anzahl der gelegten X gleich der Anzahl der entfernten X ist, also genau dann, wenn $2j = 3k$ gilt.

Zweig 2: Pruefe $i + k = j$

Fuer jedes a wird ein Symbol A auf den Stack gelegt. In der b -Phase werden zuerst diese A -Symbole entfernt. Sobald keine A -Symbole mehr uebrig sind,

wird fuer jedes weitere b ein Symbol B auf den Stack gelegt. In der c -Phase wird fuer jedes c ein B entfernt. Am Ende wird akzeptiert, wenn genau der Bodenmarker erreicht ist.



Dieser Zweig akzeptiert genau dann, wenn die Zahl der b 's ausreicht, um zuerst alle i Symbole fuer die a 's zu entfernen, und danach genau k weitere b 's fuer die c 's gespeichert wurden. Also gilt genau

$$j = i + k.$$

Da der PDA zu Beginn nichtdeterministisch einen der beiden Zweige waehlt, akzeptiert er genau dann, wenn mindestens eine der Bedingungen erfuehrt ist. Somit gilt

$$L(P) = \{a^i b^j c^k \mid 2j = 3k \vee i + k = j\}.$$